

High level specifications for project on Agentic AI and Model fine-tuning
CCAI 2025-26
Prof. Daniela Giordano

The project integrates into a single scenario (Blogger support) the three main competences of the course: building an agentic architecture, using a semantic component (knowledge graph) and fine tuning a model. The project is 80% of the grade.

Goal:

Design, implement and evaluate an Agentic AI system (a “Copilot for domain-specific task”) in the following scenario

Scenario:

Blogger Support

You are a blogger writing in the domain of your choice (e.g. your profession, your hobbies, your interests...). You aim at establishing your blog as a source of trusted information and knowledge.

You plan to write every (n) day(s) a post. The post should be an article (you set the max length) that can address one of the following:

- Upcoming events in your town/region related to your topics
- “How to” posts. These are posts in which you share with your readers a selected resource (e.g. an article) about how to do a certain thing. Example: your blog is about sailing and you want to write a post about how to do winch maintenance.
- A review of a new product/technology
- Interesting advances/news related to the domain

Your task is to prototype an Agentic system with LangGraph to support you in your activity.

The Agentic system should have (at least) the following capabilities:

- Verify the accuracy of the information gathered in Internet
- Selects among the resources found in Internet, based on some notion of quality and interestingness
- Drafts a post for you to review (Note: implementation of human-in the loop is required)
- Suggests you the topics of possible posts.
- Keep memory of previous posts and suggest topics that have not been addressed recently.
- The agents plans a sequence of posts and justifies its choices, and proceeds to prepare them.

Note: writing the prepared posts to an external DB is not required

Knowledge Graph Requirements

The system must maintain an **editorial Knowledge Graph** representing the evolving knowledge of the blog.

The Knowledge Graph must include at least:

- previously generated posts
- topics covered
- sources used
- key claims extracted from the content
- relationships between topics

The Knowledge Graph must be **actively used**, not only stored.

In particular, it must be queried at least in the following stages:

1. **Topic suggestion phase**
 - to avoid repetition
 - to identify gaps in coverage
2. **Post drafting phase**
 - to ensure consistency with previously published content
 - to connect the new post with existing topics

The Knowledge Graph must be **incrementally updated** after each approved post.

K-RAG (Knowledge Graph–augmented RAG) Requirements

The system must implement a **Knowledge Graph–augmented Retrieval-Augmented Generation (K-RAG)** approach.

This includes:

- document retrieval from external or local sources (RAG)
- integration of Knowledge Graph information into the retrieval process
- use of both structured knowledge (KG) and unstructured documents (RAG)

In particular, the system must:

- use the Knowledge Graph to **expand or refine retrieval queries**
- use retrieved documents to **support claims in the generated post**
- explicitly include **citations or references** in the generated content

The final output must clearly demonstrate that:

- the content is grounded in retrieved documents
- the reasoning is informed by the Knowledge Graph

Reasoning and Tool Usage

The system must implement an **agentic reasoning process** (e.g., ReAct-style).

This requires:

- explicit reasoning steps (e.g., Thought → Action → Observation)
- dynamic selection of tools based on the task
- justification of tool usage in the reasoning trace

The system must include at least the following tools:

- a **Search tool** for retrieving external information
- a **RAG retrieval tool** for document-based knowledge
- a **Knowledge Graph tool** for querying and updating structured knowledge

The system must also include **at least two additional tools** designed and implemented by the team
Note: one of this tool must be based on the fine-tuned model

Planning Requirements

The system must include a **planning capability** that:

- generates a sequence of blog posts
- justifies the order and selection of topics
- ensures diversity and coverage of the domain

The planning process must:

- take into account previously generated content (via the Knowledge Graph)
- avoid redundancy
- demonstrate coherent editorial strategy

Model Context Protocol (MCP) / State Management

The system must explicitly manage its internal state (context), including:

- user input
- reasoning trace
- tool outputs
- Knowledge Graph summaries
- planning information

The state must be:

- updated at each step of the agent execution
 - used as input for subsequent reasoning steps
-

Human-in-the-loop Requirement

The system must include a **human-in-the-loop mechanism** for reviewing generated content.

Before updating the Knowledge Graph, the system must:

- present the generated post to the user
- allow the user to:
 - approve
 - modify
 - reject and regenerate

The Knowledge Graph must be updated **only after user approval**.

Tooling Requirements (specific)

The system must:

- use at least three core tools:
 - Search tool
 - RAG retrieval tool
 - Knowledge Graph tool
 - implement at least **one** additional tool designed by the team and **one** based on the fine-tuned model
 - select tools dynamically through the reasoning process
 - provide a clear justification for each tool invocation
-

Guidelines for Fine-Tuning a Component in an Agentic AI Pipeline

1. **Problem definition within the agentic pipeline.** Which component of the pipeline do you want to develop or fine-tune? What input does the component receive? What output should it produce? How is this output used in the rest of the agentic pipeline?
2. **Definition of a baseline.** Define a simple baseline to compare against the fine-tuned model. The baseline can be a heuristic rule or a model used without fine-tuning.
3. **Definition of evaluation metrics.** Choose one or more metrics to evaluate the component. The metrics must be consistent with the problem being addressed.
4. **Definition of the dataset.** Choose a dataset suitable for the problem, from Hugging Face, other public sources, or generated data. Explain why the dataset is suitable for the selected component and describe the format of the examples, namely the expected input and output.
5. **Dataset split.** Split the dataset into train, validation, and test sets. The train set is used to train the model, the validation set is used to select the model and hyperparameters, and the test set is used only for the final evaluation.
6. **Training/Fine-tuning/Prompt-tuning.** Specify which technique is adopted, for example full fine-tuning, LoRA, or another technique suitable for the task.

7. **Model selection.** Test multiple configurations by varying, for example, learning rate, batch size, number of epochs, starting model, or parameters of the chosen fine-tuning technique. Design choices can also be tested here. The goal is to understand which choices have the strongest impact on the evaluation metric, measured on the validation set.
8. **Final comparison with the baseline.** Compare the fine-tuned model with the baseline using the metrics defined at the beginning. It must be clear whether fine-tuning produced an improvement and how relevant this improvement is.
9. **Analysis of the results.** Comment on the results obtained, also showing some examples of successes and errors. The goal is to understand not only how well the model works, but also where it fails and what its limitations are.

Minimum Requirements:

- The project must focus on at least one component of the agentic pipeline.
- At least one simple baseline must be defined to compare against the developed model.
- At least one evaluation metric consistent with the task must be selected.
- A dataset suitable for the problem must be used, specifying its source, format, and expected target.
- The dataset must be split into train, validation, and test sets.
- Training, fine-tuning, or prompt-tuning of a model must be performed, briefly motivating the chosen technique.
- A hyperparameter search must be carried out, within the available time, using the validation set.
- The final model must be compared with the baseline on the test set.
- The code must be understandable and fully explainable by the group.
- Produce documentation (following guidelines' steps) e.g. a presentation

Optional but Strongly Recommended:

Do not limit the work to hyperparameter search only. Particular emphasis should be placed on the process: error analysis; improvement hypothesis; solution modification; new evaluation; comparison with the baseline and the previous model.

Examples of design choices to test:

- Adding terms to the loss function.
- Expanding the dataset, for example by generating synthetic data for the most difficult cases.
- Performing additional pre-training on domain-specific data before fine-tuning.
- Modifying the input/output format, for example by adding context or making the output more structured.
- Comparing different strategies, such as zero-shot, few-shot, prompt-tuning, LoRA, and full fine-tuning.
- Adding new information or input modalities, if useful for the task.
- ...

Golden Rule: show experimental reasoning: do not just try random configurations

Evaluation Requirement

The system must be evaluated through:

- qualitative analysis of generated posts
- assessment of source quality and grounding
- identification of at least three failure cases
- in terms of observability parameters available in Langsmith (or similar tool).

Deliverable Requirement

The system must be accompanied by a detailed report (can be a PDF or a PPT presentation, it is up to you) in which you describe the design choices for each part of your architecture, the problems you have encountered and how you have solved them, and the experimental/evaluation activities.

For the model fine-tuning create a specific section within the document in which you explain your goal, the model you have selected, the dataset and approach used for fine tuning and the results obtained. Be prepared to justify the metrics.

The project code and the report must be uploaded in Studium 2 days before the exam

Oral discussion of the Project.

The discussion will include: the architectural choices and their justification (you can use your report or PPT presentation), commenting portions of code of the agentic architecture and portions of code regarding the fine tuning of the model.

Question on theory of the course (20%)

- 1 question if you have done the article presentation
- 2 questions if you have not done the article presentation

Suggested study resources for project development

Build a custom RAG agent with LangGraph

- self-rag (https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_self_rag/#setup)

Project: Ambient Agents with LangGraph

<https://academy.langchain.com/courses/ambient-agents>

Project: Deep research with LangGraph (primi 2 notebook)

<https://academy.langchain.com/courses/deep-research-with-langgraph>

Fine Tuning the Model: slides available in Studium